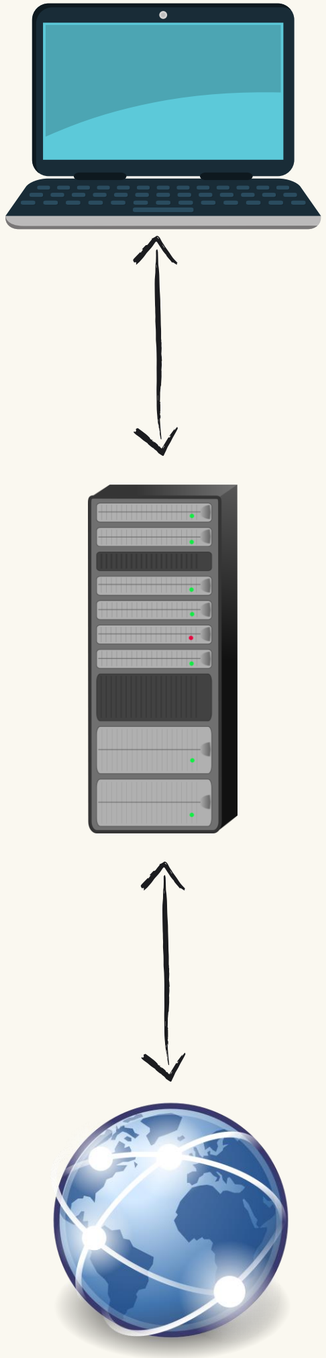
Azure Kubernetes Service (AKS) with HTTP Proxy



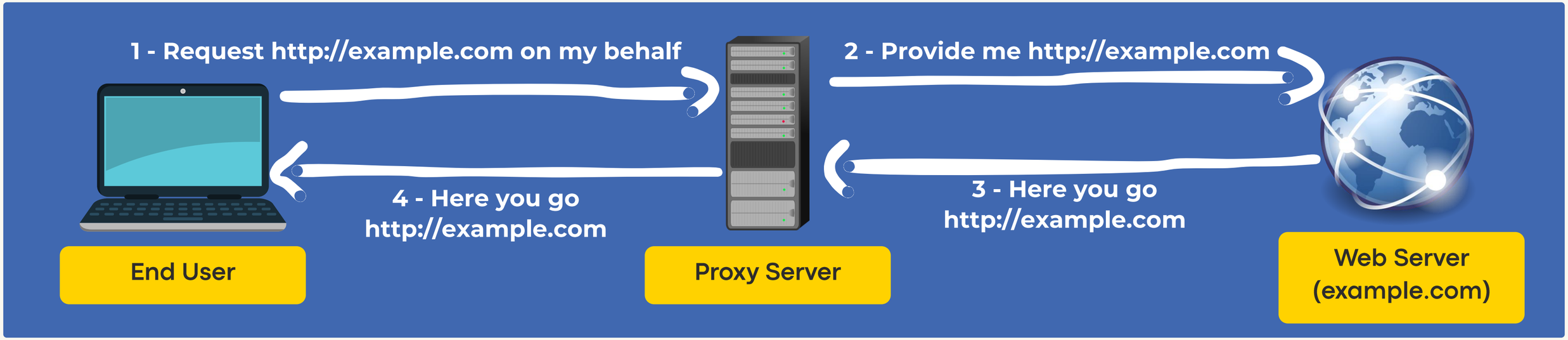
Today, we'll learn and cover:

1. What an HTTP Proxy is
2. Why an HTTP Proxy is useful
3. How AKS works with HTTP Proxy
4. In-depth demo of using AKS with HTTP Proxy (mitmproxy)
5. Explore AKS with HTTP Proxy
6. Troubleshoot AKS with HTTP Proxy

1. What an HTTP Proxy is

An HTTP proxy is a server that acts as an intermediary between a client and a web server, routing client requests and responses.

Example flow:



2. Why an HTTP Proxy is useful

Because it provides:

Better performance

Proxies cache, reduce data, and lower latency for efficiency.

Enhanced privacy and security

Proxies hide IP(s), location, enhance privacy, and block malicious sites.

Traffic filtering

Proxies filter content, restrict sites, and scan for security threats.

Traffic monitoring

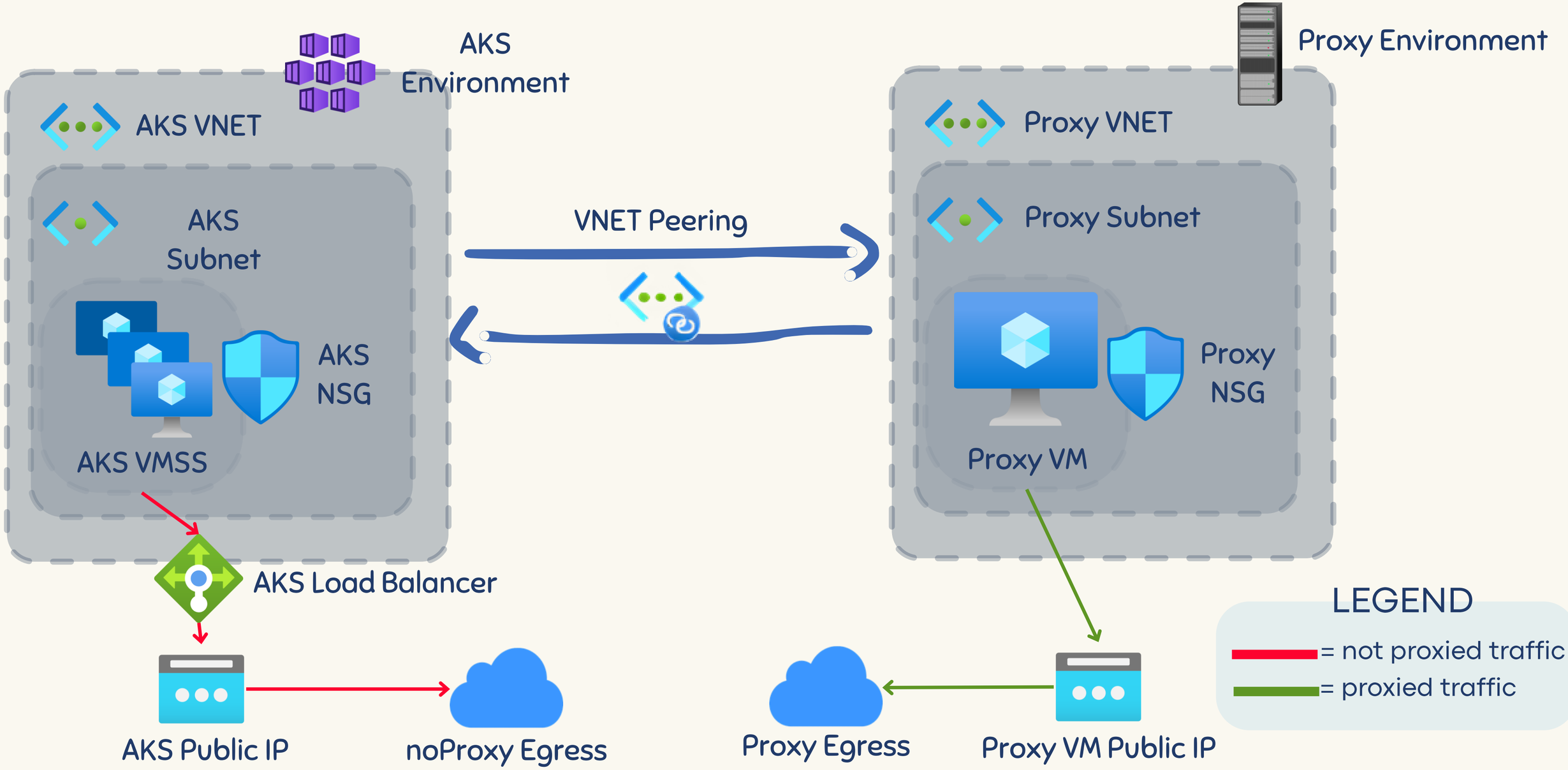
Proxies track, analyze data flow, and provide insights for efficient network management.

3. How AKS works with HTTP Proxy

Feature considerations:

- AKS doesn't provide an HTTP proxy. You need to have and configure one
- AKS requires outbound connectivity to specific dependencies like mcr.microsoft.com and others
- AKS will exclude its needed domains, IPs and CIDRs from proxying
- You can exclude domains, IPs, and CIDRs from proxying

3. How AKS works with HTTP Proxy



3. How AKS works with HTTP Proxy

The HTTP Proxy support can be enabled only during the AKS cluster creation.

```
az aks create -n $clusterName -g $resourceGroup --http-proxy-config aks-proxy-config.json
```

The HTTP Proxy configuration can be updated later.

```
az aks update -n $clusterName -g $resourceGroup --http-proxy-config aks-proxy-config-2.json
```

3. How AKS works with HTTP Proxy

aks-proxy-config.json example:

```
{
  "httpProxy": "http://myproxy.server.com:8080/",
  "httpsProxy": "https://myproxy.server.com:8080/",
  "noProxy": [
    "localhost",
    "127.0.0.1"
  ],
  "trustedCA":
"LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUgvVENDQmVXZ0F3SUJB...b3Rpbk15RGszaWFyCkYxMFls
cWNPbWVYMXVGbUtiZGkvWG9yR2xrQ29NRjNURHg4cm1wOURCaUIvCi0tLS0tRU5EIENFUlRJRklDQVRFLS0tLS0=
"
}
```

3. How AKS works with HTTP Proxy

Considerations regarding the HTTP Proxy feature enablement:

!

The pods an nodes will be injected with the following environment variables:

- HTTP_PROXY
- http_proxy
- HTTPS_PROXY
- https_proxy
- NO_PROXY
- no_proxy

!

To prevent the injection of the proxy environment variables for the pods, annotate them with:

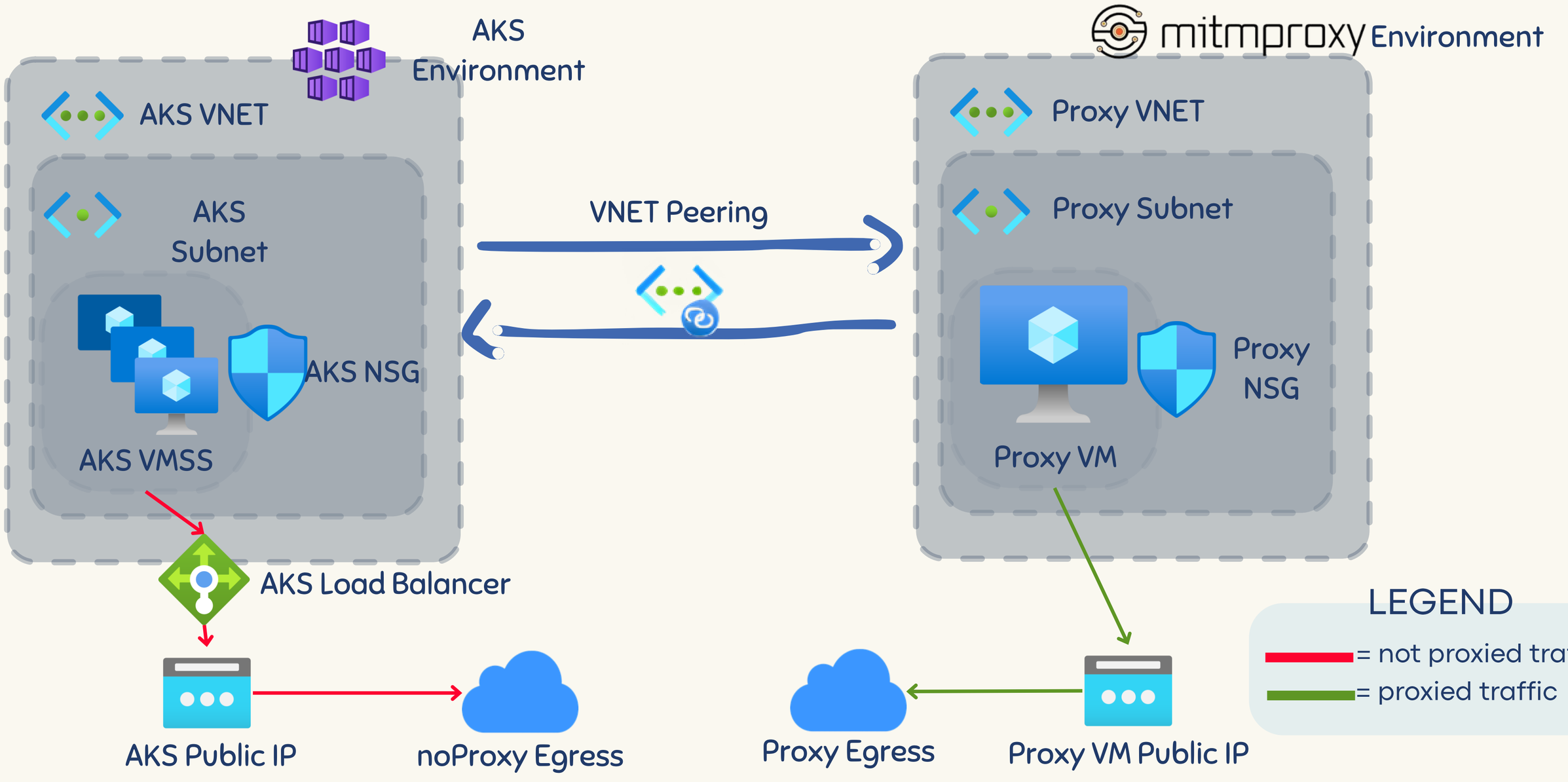
"kubernetes.azure.com/no-http-proxy-vars":"true"

3. How AKS works with HTTP Proxy

Considerations regarding the update HTTP Proxy configuration:

- httpProxy, httpsProxy, noProxy, and trustedCa values can be updated
- Rotate pods to update environment variables after the HTTP Proxy configuration operation
- After the HTTP Proxy configuration operation, a node image upgrade is needed for Kubernetes components like the nodes and containerd to have the changes
- Node(s) in new node pool(s) will have the new configuration

4. In-depth demo of using AKS with HTTP Proxy (mitmproxy)



5. Explore AKS with HTTP Proxy

Explore the pod injected environment variables:

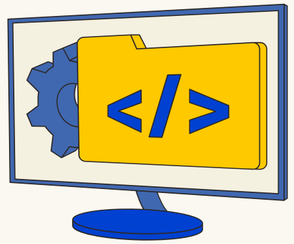
```
kubectl describe pod <pod-name> -n <namespace>
```

```
    Environment:
      no_proxy:     127.0.0.1,168.63.129.16,10.0.0.0/16,169.254.169.254,aks-proxy-aks-proxy-rg-baa1ed-4k4kx8ay.hcp.westeurope.azmk8s.io,192.168.0.0/16,
localhost,docker.io,konnectivity,10.244.0.0/16
      HTTP_PROXY:   http://10.0.0.4:8080/
      http_proxy:   http://10.0.0.4:8080/
      HTTPS_PROXY:  https://10.0.0.4:8080/
      https_proxy:  https://10.0.0.4:8080/
      NO_PROXY:     127.0.0.1,168.63.129.16,10.0.0.0/16,169.254.169.254,aks-proxy-aks-proxy-rg-baa1ed-4k4kx8ay.hcp.westeurope.azmk8s.io,192.168.0.0/16,
localhost,docker.io,konnectivity,10.244.0.0/16
```

```
kubectl exec -it <pod-name> -n <namespace> -- env
```

```
kubectl get pod <pod_name> -n <namespace> -o
jsonpath='{.spec.containers[].env}{"\n"}'
```

5. Explore AKS with HTTP Proxy



Explore the node configuration:

```
env #can be confusing if you got inside the node with a helper pod because it will show the environment variables of the helper pod, not of the node
```

```
cat /etc/environment #the recommended way to check node's environment variables
```

```
cat /etc/systemd/system.conf.d/proxy.conf #used by systemd services (like containerd and kubelet) to configure the HTTP Proxy behavior
```

```
cat /etc/apt/apt.conf.d/95proxy #used by the package manager to configure the HTTP Proxy behavior
```

```
openssl x509 -in /etc/ssl/certs/proxyCA.pem -noout -text #explore the proxy certificate
```

6. Troubleshoot AKS with HTTP Proxy

If AKS can't reach an endpoint, you can:

- verify proxy's configuration and logs
- isolate the issue
- update the noProxy value (remember about the need of node image upgrade / new node pool)

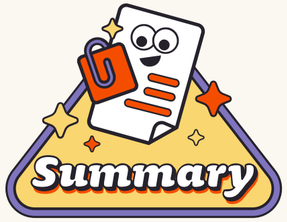
Isolate the issue by testing if the machine where the proxy is installed can reach the endpoint via the proxy:

```
curl <url> --proxy <protocol>://<proxy_address>:<proxy_port> #from proxy
#Example: curl example.com --proxy http://127.0.0.1:8080
```

Isolate the issue by bypassing the proxy from the AKS node/pod:

```
curl <url> --noproxy "*" #from AKS node/pod
#Example: curl example.com --noproxy "*"
```

Today, we've learned and covered:



- What an HTTP Proxy is and why it is useful
- Considerations of how AKS works with HTTP Proxy
- How to configure and use an HTTP proxy (mitmproxy) on a VM
- How to create and update an AKS cluster with an HTTP proxy configuration
- How to explore and troubleshoot AKS with HTTP Proxy